



Application Web orientée Services — Architecture Microservices

Master 2 MIAGE SITN · Projet Applications Web orientées Services (M. Menceur) · 2025–2026

Binôme : Djena Haddar · Jules (à compléter) | **Dépôt GitHub** : (à compléter)

1. Présentation

Le projet réalise le jeu **Motus** sous la forme d'une application **microservices Java / Spring Boot**, avec persistance **JPA / Spring Data** sur une base relationnelle et un client de démonstration. La règle : deviner un mot mystère en un nombre limité d'essais ; le joueur part de la première lettre du mot, et chaque proposition renvoie, pour chaque lettre, si elle est **bien placée**, **mal placée** ou **absente**.

L'application couvre les quatre exigences fonctionnelles du sujet :

- **Gérer les joueurs** — inscription, connexion, historique des parties.
- **Gérer les parties** — mot mystère aléatoire, propositions, calcul des lettres bien/mal placées, fin de partie.
- **Suivre le score / historiser** — enregistrement des résultats et classement global.
- **Administrer le jeu** — lister et rechercher les parties par joueur, statut, date et performance.

2. Compiler et exécuter le projet

Le projet se lance intégralement avec **Docker Compose** (un seul prérequis : Docker Desktop démarré), à la racine du dépôt :

```
docker compose up -d --build # construit + démarre les 4 services, la base, le client
# Client web : http://localhost:8090
```

Arrêt : `docker compose down` (`-v` pour effacer aussi les données). Compte administrateur créé automatiquement : `admin` / `admin123`.

Points d'accès

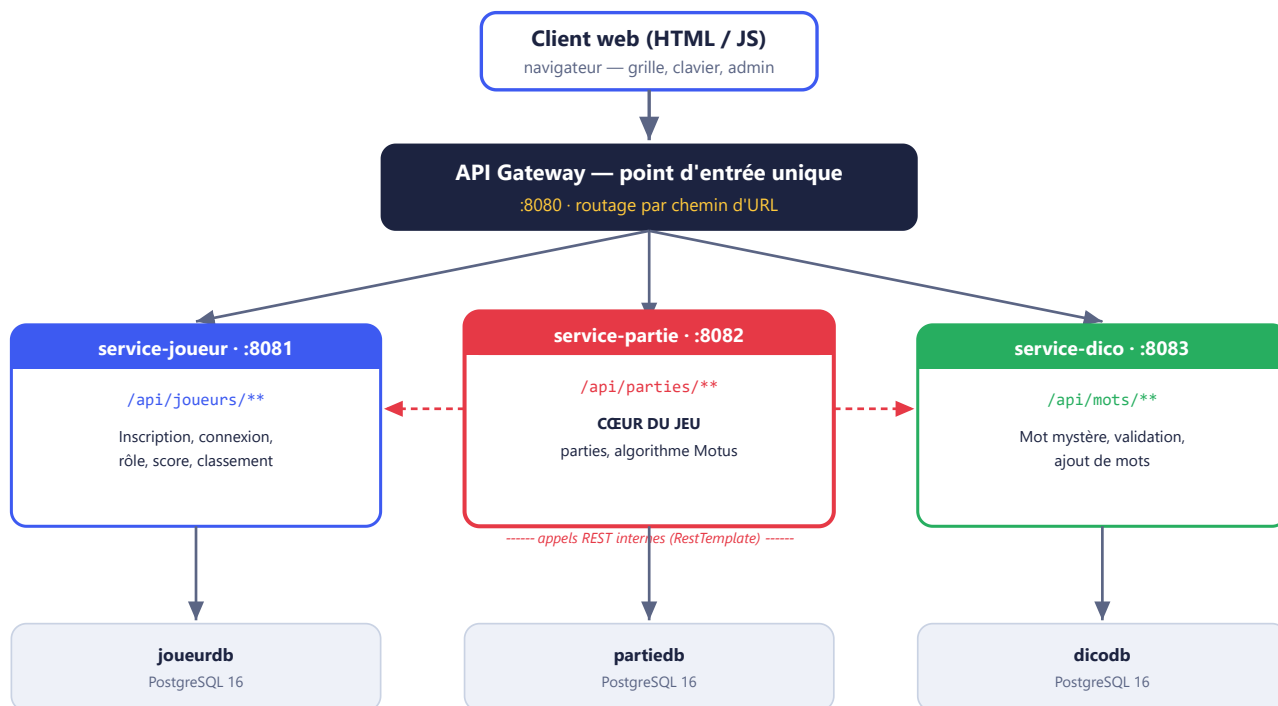
ÉLÉMENT	URL	RÔLE
Client web	http://localhost:8090	Interface de jeu
API Gateway	http://localhost:8080	Point d'entrée unique
service-joueur	:8081	Joueurs, scores, classement
service-partie	:8082	Cœur du jeu
service-dico	:8083	Dictionnaire
PostgreSQL	:5432	3 bases (motus / motus)

Logiciels et versions

COMPOSANT	VERSION	RÔLE DANS LE PROJET
Java (JDK cible)	21	Langage
Spring Boot	3.3.5	Framework microservices (Web, Data)
Spring Cloud Gateway	2023.0.3	API Gateway / routage
Spring Data JPA / Hibernate	6.5 (via Boot)	Persistance ORM
PostgreSQL	16	Base de données (production)
H2	runtime	Base en mémoire (tests)
Maven	3.9	Build et gestion des dépendances
Docker / Docker Compose	v2	Conteneurisation, orchestration locale

3. Architecture

L'application est découpée en **trois microservices métier** derrière une **API Gateway** (point d'entrée unique), avec **une base PostgreSQL par service**. Chaque service possède exclusivement ses données : c'est le principe de couplage faible propre aux microservices. Le **service-partie** orchestre les deux autres par appels **REST** internes.



Une base isolée par microservice — aucune clé étrangère entre services

Figure 1 — Architecture microservices et flux de communication

Communication entre services

- **service-partie** → **service-dico** : tirage du mot mystère et validation des propositions.
- **service-partie** → **service-joueur** : historisation du résultat (**tolérante à la panne** : un échec ne bloque pas la partie) et vérification du rôle administrateur (**stricte** : refus par défaut en cas de doute).

4. API REST

Chaque microservice expose une API REST sur sa ressource. Le client ne dialogue qu'avec la Gateway, qui relaie vers le service correspondant selon le chemin.

SERVICE	VERBE	CHEMIN	RÔLE
service-dico	GET	/api/mots/aleatoire?longueur=N	Tire un mot mystère de N lettres
	GET	/api/mots/existe?valeur=MOT	Vérifie l'existence d'un mot
	POST	/api/mots	Ajoute un mot au dictionnaire (admin)
service-joueur	POST	/api/joueurs	Inscrit un joueur
	POST	/api/joueurs/connexion	Connexion (pseudo + mot de passe)
	GET	/api/joueurs/classement	Classement par score décroissant
	GET	/api/joueurs/{id}	Consulte un joueur et son historique
	POST	/api/joueurs/{id}/resultats	Historise un résultat, met à jour le score
	GET	/api/joueurs/{id}/est-admin	Contrôle de rôle (appel interne)
service-partie	POST	/api/parties	Crée une partie (mot mystère aléatoire)
	GET	/api/parties?adminId=...	Recherche des parties (admin, filtres)
	POST	/api/parties/{id}/propositions	Soumet une proposition de mot
	POST	/api/parties/{id}/abandon	Abandonne la partie (→ PERDUE)

Sécurité de la donnée. Le mot mystère n'est **jamais** renvoyé au client tant que la partie est **EN_COURS** : seule sa première lettre est exposée (un objet de transfert dédié, **PartieVue**, filtre la réponse). Le mot complet n'apparaît qu'une fois la partie gagnée ou perdue. Cela empêche de lire la solution dans la réponse JSON.

Gestion uniforme des erreurs

Chaque service traduit ses exceptions métier en codes HTTP explicites via un gestionnaire global (**@RestControllerAdvice**) : **400** (coup invalide), **403** (accès refusé — réservé aux administrateurs), **404** (ressource introuvable), **409** (pseudo déjà pris), **503** (service indisponible). Le corps de réponse est un JSON homogène (**{ statut, erreur, message, timestamp }**).

5. Diagramme de classes métier

Les entités JPA se répartissent sur les trois bases. Au sein d'un service, les relations sont de vraies associations JPA (`@OneToMany`). Entre services, les liens (`joueurId` , `partieId`) sont de simples **références logiques** : chaque base ignore les entités des autres services, il n'existe donc aucune clé étrangère inter-bases.

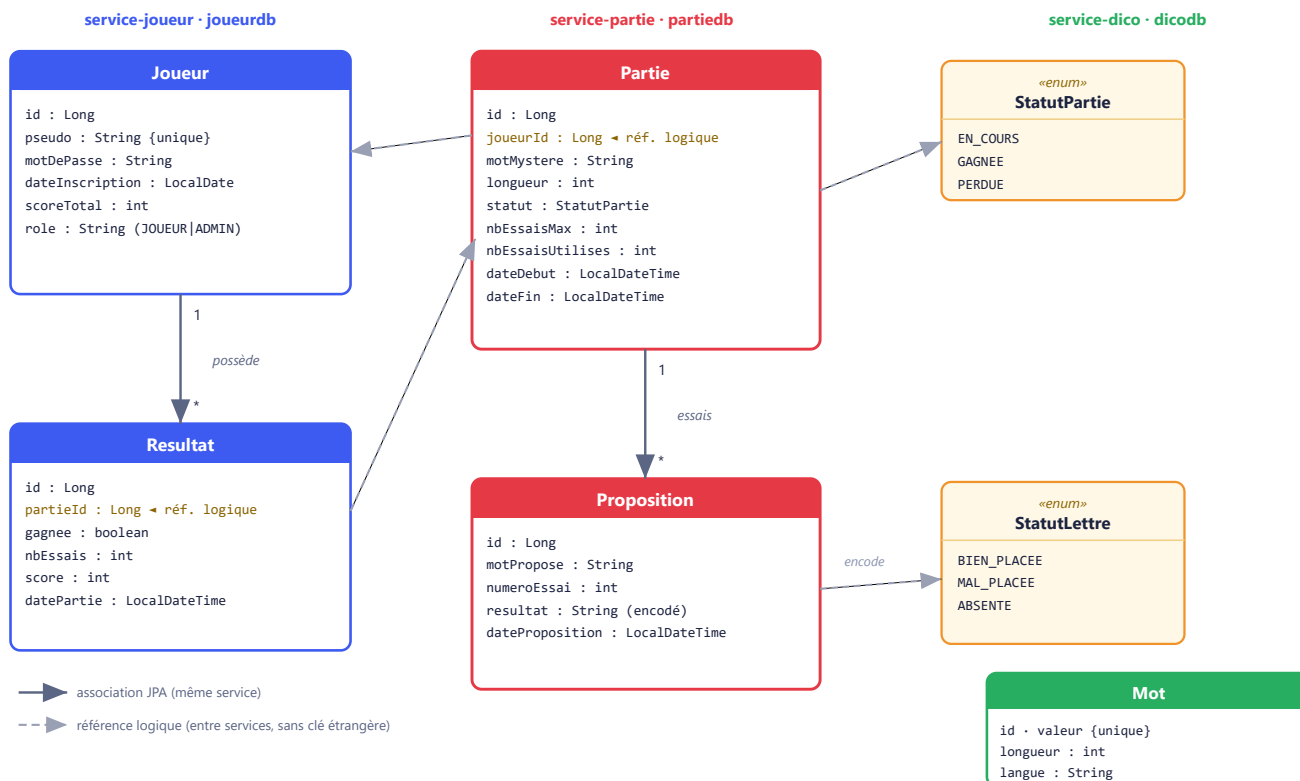


Figure 2 — Entités métier, associations JPA et références logiques inter-services

6. L'algorithme Motus

Le cœur du jeu (classe `AlgorithmeMotus`) compare le mot proposé au mot mystère et attribue à chaque lettre l'un des trois statuts. La difficulté est la **gestion des lettres en double** : une approche naïve (« la lettre est-elle dans le mot ? ») marquerait trop de lettres « mal placées ». L'algorithme procède donc **en deux passes**, à l'aide d'un compteur d'occurrences des lettres du mot mystère :

- 1^{re} **passse — bien placées**. On parcourt les positions : si la lettre proposée est identique à celle du mot mystère à la même position, elle est marquée `BIEN_PLACEE` et on **décompte** une occurrence de cette lettre.
- 2^e **passse — mal placées / absentes**. Pour les lettres restantes : s'il reste encore une occurrence disponible de la lettre dans le compteur, elle est `MAL_PLACEE` (et on décompte cette occurrence) ; sinon elle est `ABSENTE`.

Grâce au décompte, une même lettre n'est jamais signalée plus de fois qu'elle n'apparaît réellement dans le mot mystère. Exemple : mot mystère `ARBRE`, proposition `SERRE` — le second `R` et le `E` final sont correctement départagés au lieu d'être tous deux marqués présents. Le résultat est encodé en chaîne (ex. `"ABSENTE,MAL_PLACEE,ABSENTE,BIEN_PLACEE,BIEN_PLACEE"`) et stocké avec la proposition.

Score et classement

À la fin d'une partie, **service-partie** transmet le résultat à **service-joueur**, qui calcule le score et mesure la performance. Une partie gagnée rapporte `100 - 10 × nbEssais` points (minimum 10) ; une partie perdue rapporte 0. Gagner en peu d'essais rapporte donc davantage, ce qui donne du sens au classement (trié par score décroissant, comptes administrateur exclus).

7. Bilan du projet

Ce que nous avons appris

- Le découpage en microservices : une base par service, références logiques plutôt que clés étrangères.
- Le rôle d'une **API Gateway** comme unique point d'entrée et façade de routage.
- La communication REST inter-services et la **tolérance aux pannes** (dégradation gracieuse).
- L'intérêt des objets de transfert (DTO) pour ne pas exposer l'état interne (mot mystère masqué).

Réussites

- Une stack complète lancée en une commande (`docker compose up`).
- Un algorithme Motus correct sur les lettres en double, couvert par des tests.
- Une séparation nette contrôleur / service / repository et une gestion d'erreurs homogène.

Difficultés

- La gestion des doublons dans l'algorithme (d'où l'approche en deux passes).
- La configuration réseau entre conteneurs : joindre un service par son nom plutôt que par `localhost`, via des variables d'environnement surchargées au déploiement.
- La coordination du contrôle d'accès administrateur, dont la donnée « rôle » n'appartient qu'à **service-joueur**.

Démarche qualité

- Tests d'intégration (`MockMvc`, base H2, clients REST simulés) sur les quatre services.
- Revue de code avant soutenance : suppression des méthodes et endpoints inutilisés et de la logique résiduelle, pour un code au plus près du strict nécessaire.

Perspectives

- Compléter le déploiement Kubernetes / MiniKube (base et gateway).
- Chiffrer les mots de passe et introduire une authentification par jeton.